MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics

REPORT

Laboratory Work No. 3

Topic: **Polyalphabetic Ciphers**

Task 3.1 – the Playfair cipher for the Romanian alphabet

Author: Bîtca Cătălin
Group: FAF-242

Chișinău, 2026

1. Short theory

1.1. Polyalphabetic ciphers

In Lab 2 I saw that a monoalphabetic cipher is easy to break, because every letter is always replaced by the same letter and the frequencies of the language stay visible in the ciphertext. A cipher is stronger if its ciphertext has a flatter frequency distribution, which gives the cryptanalyst less information. Polyalphabetic ciphers do this by encrypting the same plaintext letter in different ways, depending on its position or on its neighbours. Then a frequent letter like E is spread over several cipher letters, and its high frequency is mixed with the low frequencies of other letters. The idea goes back to Leon Battista Alberti in the 15th century (his essay on ciphers was the text I decrypted in Lab 2).

The best-known example is the **Vigenère cipher**. It works like the Caesar cipher, but instead of one shift it uses several shifts k_1, k_2, …, usually given by the letters of a key word, which is repeated along the message:

$$c_i = (m_i + k_i) \bmod n, \quad m_i = (c_i - k_i) \bmod n.$$

For a key of length L there are n^L possible keys, and the attacker does not know L. In the lab, students with an even number in the group list implement Vigenère (Task 3.2) and students with an odd number implement Playfair (Task 3.1). My number is 7, so I implemented **Task 3.1, the Playfair cipher**.

1.2. The Playfair cipher

The Playfair cipher was invented by Charles Wheatstone and first described on 26 March 1854, but it carries the name of his friend Lord Lyon Playfair, who convinced the British government to use it. It was used by the British army in the Boer War, in a modified form in World War I, and by the Australian army in World War II. It is one of the first ciphers that encrypts **blocks** of text instead of single letters: it always encrypts **pairs of letters** (digraphs).

For English, the key is written into a 5×5 matrix: first the letters of the key without repetitions, then the remaining letters of the alphabet in order (J is merged with I so that 25 letters fit). Encryption has three steps:

1. **Preparing the text:** uppercase, no spaces and punctuation, split into pairs. If a pair would contain two equal letters, a rare filler letter (X, Q or Z) is inserted between them, and if the number of letters is odd, a filler is added at the end.
2. **Building the matrix** from the key.
3. **Encrypting every pair** with one of three rules:
    - the letters are in **different rows and columns**: each letter is replaced by the letter in its own row but in the column of the other letter (the opposite corners of the rectangle);

- the letters are in the **same row**: each letter is replaced by the letter to its right (the last one wraps to the first);
- the letters are in the **same column**: each letter is replaced by the letter below it (the last one wraps to the first).

Decryption uses the same matrix and the same rules in the opposite direction: in the same row the letter to the **left** is taken, in the same column the letter **above**, and the rectangle rule stays exactly the same. The decrypted text still contains the filler letters and has no spaces, so the last step (removing the fillers and putting the spaces back) is done by a human who understands the language.

1.3. Strength and weaknesses

Because Playfair works on pairs, the same plaintext letter is encrypted differently depending on the letter it is paired with, so the frequencies of single letters are much flatter than in a monoalphabetic cipher. The attacker has to work with the frequencies of pairs instead, and there are many more of them ($25 \cdot 24 = 600$ possible pairs for English), so a much longer ciphertext is needed. The number of possible English matrices is $25! \approx 1.55 \cdot 10^{25}$.

Still, Playfair is not secure by modern standards. It is a substitution of pairs, so the frequencies of pairs are preserved. It also has visible patterns: if AB is encrypted as XY, then BA is encrypted as YX; a letter is never encrypted into itself; the ciphertext always has an even length and never contains a pair of two equal letters. With these properties a computer finds the key in seconds, and a good cryptanalyst can do it by hand.

2. Adapting Playfair to the Romanian alphabet

The task is to implement Playfair for messages in Romanian, which has 31 letters (Table 1).

Letter	A	Ă	Â	B	C	D	E	F	G	H	I	Î	J	K	L	M	N	O	P	~~Q~~	R	S	Ș	T	Ț	U	V	W	X	Y	Z
Code	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30

Table 1. *The Romanian alphabet (n = 31); Q is the letter left out of the matrix*

Size of the matrix. 31 is a prime number, so the letters cannot fill a rectangle exactly. I compared the possible sizes:

- 6×6 (36 cells) or 4×8 (32 cells): all 31 letters fit, but the extra cells must be filled with other symbols (digits, signs). Those symbols would then appear in the ciphertext, and the task says that the text may contain only letters;
- 5×6 (30 cells): only letters are used, but one letter has to be left out, exactly like J in the English version.

I chose the **5×6 matrix** and left out **Q**, because it is the rarest letter in Romanian (0.00% in Table 2.1 of Lab 2; it appears only in foreign words). Every Q in the key, the message or the ciphertext is written as **K**, the letter with the same sound, just like J is written as I in the English Playfair. The lab description allows this: the rarest letter can be eliminated, and at decryption it is restored by the reader. So the matrix contains these 30 letters:

```
A Ă Â B C D E F G H I Î J K L M N O P R S Ș T Ț U V W X Y Z
```

Filler letter. Between two equal letters in a pair and at the end of an odd-length text I insert **X**, which is also very rare in Romanian (0.11%). If the letter that needs a filler is itself X, I use **Z** instead, so that the filler never creates a new double.

Key and text. The key, the message and the ciphertext may contain only Romanian letters (upper or lower case) and spaces. Spaces are removed and the letters are converted to uppercase; Ş and Ţ written with a cedilla are accepted and converted to Ș and Ț with a comma, like in Lab 1. The key must have at least 7 letters (spaces are not counted); repeated letters are then removed from it before it is written into the matrix.

Ciphertext checks. A Playfair ciphertext always has an even number of letters and never contains a pair of two equal letters, so the program refuses ciphertexts that break these rules.

Key space. There are $30! \approx 2.65 \cdot 10^{32}$ ways to fill the 5×6 matrix. Rotating the rows or the columns of the matrix cyclically gives exactly the same cipher ($5 \cdot 6 = 30$ equivalent matrices), so there are about $29! \approx 8.84 \cdot 10^{30}$ really different keys, still far too many for brute force.

3. Implementation

I wrote the program in **Python 3**, in one file, `playfair.py`. The user chooses encryption or decryption, enters the key, sees the matrix, and then enters the message or the ciphertext. For encryption the program shows the pairs and the ciphertext; for decryption it shows the decrypted pairs and the text without spaces, which is then completed by hand, as the task says.

3.1. Alphabet and letter conversion

```
import unicodedata

ALPHABET = "AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ"
LOWERCASE = "aăâbcdefghiîjklmnopqrsștțuvwxyz"
MATRIX_ALPHABET = ALPHABET.replace("Q", "")
COLUMNS = 6
MIN_KEY_LENGTH = 7
LETTERS = "Romanian letters (A–Z, a–z, Ă, Â, Î, Ș, Ț)"

UPPER_OF = {}
for i in range(len(ALPHABET)):
    UPPER_OF[ALPHABET[i]] = ALPHABET[i]
    UPPER_OF[LOWERCASE[i]] = ALPHABET[i]
UPPER_OF["Ş"] = "Ș"
UPPER_OF["ş"] = "Ș"
UPPER_OF["Ţ"] = "Ț"
UPPER_OF["ţ"] = "Ț"
```

`ALPHABET` is the Romanian alphabet in the order of Table 1, and `MATRIX_ALPHABET` is the same alphabet without Q (30 letters), which fills a matrix with 6 columns and 5 rows. `UPPER_OF` maps every allowed character (upper or lower case) to its uppercase letter; it also maps the cedilla letters Ş, ş, Ţ, ţ (U+015E, U+015F, U+0162, U+0163) to the correct Ș and Ț. A character that is not in this dictionary is not allowed.

3.2. Preparing the text

```
def prepare_text(text):
    text = unicodedata.normalize("NFC", text)
    result = ""
    for ch in text:
        if ch == " ":
            continue
        if ch not in UPPER_OF:
            raise ValueError(ch)
        result += UPPER_OF[ch]
    return result.replace("Q", "K")
```

The text is normalized to Unicode NFC (so that a letter like Ă typed as A plus a separate breve becomes one character), spaces are skipped and every letter is converted to uppercase. The first character that is not allowed stops the function with a `ValueError` that carries that character, so the program can show it to the user. At the end every Q is replaced by K, because Q is not in the matrix. The same function is used for the key, the message and the ciphertext.

3.3. Building the matrix

```
def build_matrix(key, alphabet=MATRIX_ALPHABET, columns=COLUMNS):
    letters = ""
    for ch in key + alphabet:
        if ch in alphabet and ch not in letters:
            letters += ch
    return [letters[i:i + columns] for i in range(0, len(letters), columns)]
```

The function goes through the key followed by the whole alphabet and keeps every letter only the first time it appears, so the repeated letters of the key disappear and all remaining letters follow in alphabetical order. The resulting 30-letter string is cut into rows of 6 letters. The alphabet and the number of columns are parameters, so the same function can also build the 5×5 English matrix from the lab description, which I used to check my program on the example given there.

3.4. Splitting the text into pairs

```
def make_pairs(text):
    pairs = []
    i = 0
    while i < len(text):
        first = text[i]
        if i + 1 < len(text) and text[i + 1] != first:
            pairs.append(first + text[i + 1])
            i += 2
        else:
            pairs.append(first + ("Z" if first == "X" else "X"))
            i += 1
    return pairs
```

The text is read from left to right. If the next letter exists and is different from the current one, both letters form a pair and the index moves by 2. Otherwise (two equal letters, or the last letter of an odd text) the current letter gets the filler X (or Z if the letter is X) and the index moves only by 1, so the second equal letter starts the next pair. For example COPIII becomes CO PI IX IX.

3.5. Encrypting or decrypting one pair

```
def transform_pair(pair, matrix, step):
    position = {letter: (r, c) for r, row in enumerate(matrix) for c, letter in
enumerate(row)}
    (r1, c1), (r2, c2) = position[pair[0]], position[pair[1]]
    rows, columns = len(matrix), len(matrix[0])
    if r1 == r2:
        return matrix[r1][(c1 + step) % columns] + matrix[r2][(c2 + step) % columns]
    if c1 == c2:
        return matrix[(r1 + step) % rows][c1] + matrix[(r2 + step) % rows][c2]
    return matrix[r1][c2] + matrix[r2][c1]
```

This is the core of the algorithm. First it builds a dictionary from every letter to its (row, column) in the matrix. Then it applies the three rules. The parameter `step` is **+1** for encryption (letter to the right or below) and **−1** for decryption (letter to the left or above); the `%` operation makes the last letter of a row or column wrap around to the first one and back. The rectangle rule does not depend on the direction, because swapping the columns twice gives back the original pair, so one function does both encryption and decryption.

3.6. Encryption and decryption

```
def encrypt(text, matrix):
    return [transform_pair(pair, matrix, 1) for pair in make_pairs(text)]


def decrypt(text, matrix):
    pairs = [text[i:i + 2] for i in range(0, len(text), 2)]
    return [transform_pair(pair, matrix, -1) for pair in pairs]
```

`encrypt` splits the prepared message into pairs (with the fillers) and encrypts every pair with `step = 1`. `decrypt` cuts the ciphertext into pairs of two letters and transforms them with `step = -1`. Both return the list of pairs, so the program can show them separated by spaces and also joined together.

3.7. Input validation

```
def ciphertext_error(text):
    if len(text) % 2 == 1:
        return f"The ciphertext must have an even number of letters (it has {len(text)})."
    for i in range(0, len(text), 2):
        if text[i] == text[i + 1]:
            return f"The pair '{text[i:i + 2]}' (letters {i + 1}–{i + 2}) cannot appear in a
Playfair ciphertext."
    return None
```

```
def read_key():
    while True:
        key = read_text(f"Key (at least {MIN_KEY_LENGTH} letters): ")
        if len(key) >= MIN_KEY_LENGTH:
            return key
        print(f"Key too short ({len(key)} letters). It must have at least {MIN_KEY_LENGTH}
letters.")


def read_ciphertext():
    while True:
        text = read_text("Ciphertext: ")
        error = ciphertext_error(text)
        if not error:
            return text
        print(error)
```

`ciphertext_error` checks the two properties of every Playfair ciphertext: the number of letters is even and no pair contains two equal letters; it returns a message that explains the problem, or `None`. `read_key` and `read_ciphertext` repeat the question until the input is valid. Both use `read_text`, which rejects any character outside the Romanian alphabet (showing the character and the allowed range) and an empty text; `read_key` also requires at least 7 letters.

3.8. Main program

```
matrix = build_matrix(read_key())
print_matrix(matrix)
if option == "1":
    text = read_text("Message: ")
    print("Pairs:", " ".join(make_pairs(text)))
    print("Ciphertext:", "".join(encrypt(text, matrix)))
else:
    text = read_ciphertext()
    pairs = decrypt(text, matrix)
    print("Decrypted pairs:", " ".join(pairs))
    print("Decrypted text:", "".join(pairs))
```

This is the part of `main()` after the operation is chosen: the key is read, the matrix is built and printed, and then the message is encrypted or the ciphertext is decrypted. The menu repeats until the user chooses 0.

4. Results

4.1. The matrix

For all the examples I used the key **“criptografie”**. Without the repeated letters it becomes `CRIPTOGAFE` (10 letters, the second R and I are removed), and the other 20 letters follow in alphabetical order (Table 2).

	0	1	2	3	4	5
0	**C**	**R**	**I**	**P**	**T**	**O**
1	**G**	**A**	**F**	**E**	**Ă**	**Â**
2	**B**	**D**	**H**	**Î**	**J**	**K**
3	**L**	**M**	**N**	**S**	**Ș**	**Ț**
4	**U**	**V**	**W**	**X**	**Y**	**Z**

Table 2. *The 5×6 matrix for the key “criptografie” (key letters highlighted, rows and columns numbered from 0)*

4.2. Encryption

I encrypted the message **“Toți copiii pleacă mâine la munte”** (“All the children are leaving for the mountains tomorrow”). It was chosen because it uses all the rules: it contains Ț, Ă and Â, and three I in a row, which need a filler letter. Its pairs also cover all three encryption rules.

```
lab3 — python3 playfair.py

$ python3 playfair.py
Playfair cipher for the Romanian alphabet (5 x 6 matrix, Q is written as K)

1 – Encrypt
2 – Decrypt
0 – Exit
Choose: 1
Key (at least 7 letters): criptografie
Matrix:
   C R I P T O
   G A F E Ă Â
   B D H Î J K
   L M N S Ș Ț
   U V W X Y Z
Message: Toți copiii pleacă mâine la munte
Pairs: TO ȚI CO PI IX IP LE AC ĂM ÂI NE LA MU NT EX
Ciphertext: OCNORCTPPWPTSGGRAȘFOSFMGLVȘIÎP

1 – Encrypt
2 – Decrypt
0 – Exit
Choose: 0
```

Figure 1. *Encryption of “Toți copiii pleacă mâine la munte” with the key “criptografie”*

After removing the spaces the message is `TOȚICOPIIIPLEACĂMÂINELAMUNTE`. In `COPIII` the pair “II” is split by the filler X (`IX`), and since the text then has an odd number of letters, an X is

also added at the end (EX). The pairs are

TO ȚI CO PI IX IP LE AC ĂM ÂI NE LA MU NT EX

and the ciphertext is **OCNORCTPPWPTSGGRAȘFOSFMGLVȘIÎP**. Table 3 checks every pair by hand with the matrix from Table 2.

Pair	Positions (row, column)	Rule	Result
TO	(0,4) (0,5)	same row → letter to the right	**OC**
ȚI	(3,5) (0,2)	rectangle → swap columns	**NO**
CO	(0,0) (0,5)	same row → letter to the right	**RC**
PI	(0,3) (0,2)	same row → letter to the right	**TP**
IX	(0,2) (4,3)	rectangle → swap columns	**PW**
IP	(0,2) (0,3)	same row → letter to the right	**PT**
LE	(3,0) (1,3)	rectangle → swap columns	**SG**
AC	(1,1) (0,0)	rectangle → swap columns	**GR**
ĂM	(1,4) (3,1)	rectangle → swap columns	**AȘ**
ÂI	(1,5) (0,2)	rectangle → swap columns	**FO**
NE	(3,2) (1,3)	rectangle → swap columns	**SF**
LA	(3,0) (1,1)	rectangle → swap columns	**MG**
MU	(3,1) (4,0)	rectangle → swap columns	**LV**
NT	(3,2) (0,4)	rectangle → swap columns	**ȘI**
EX	(1,3) (4,3)	same column → letter below	**ÎP**

Table 3. *Encryption of every pair: positions in the matrix, rule used and result*

For example, T and O are both in row 0, so each is replaced by the letter to its right: T → O and O → C (O is the last letter in the row, so it wraps to the first one). E and X are both in column 3, so each is replaced by the letter below: E → Î and X → P (X is in the last row, so it wraps to the top). Ț (3,5) and I (0,2) form a rectangle, so Ț takes the letter of its row in the column of I, which is N, and I takes the letter of its row in the column of Ț, which is O.

4.3. Decryption

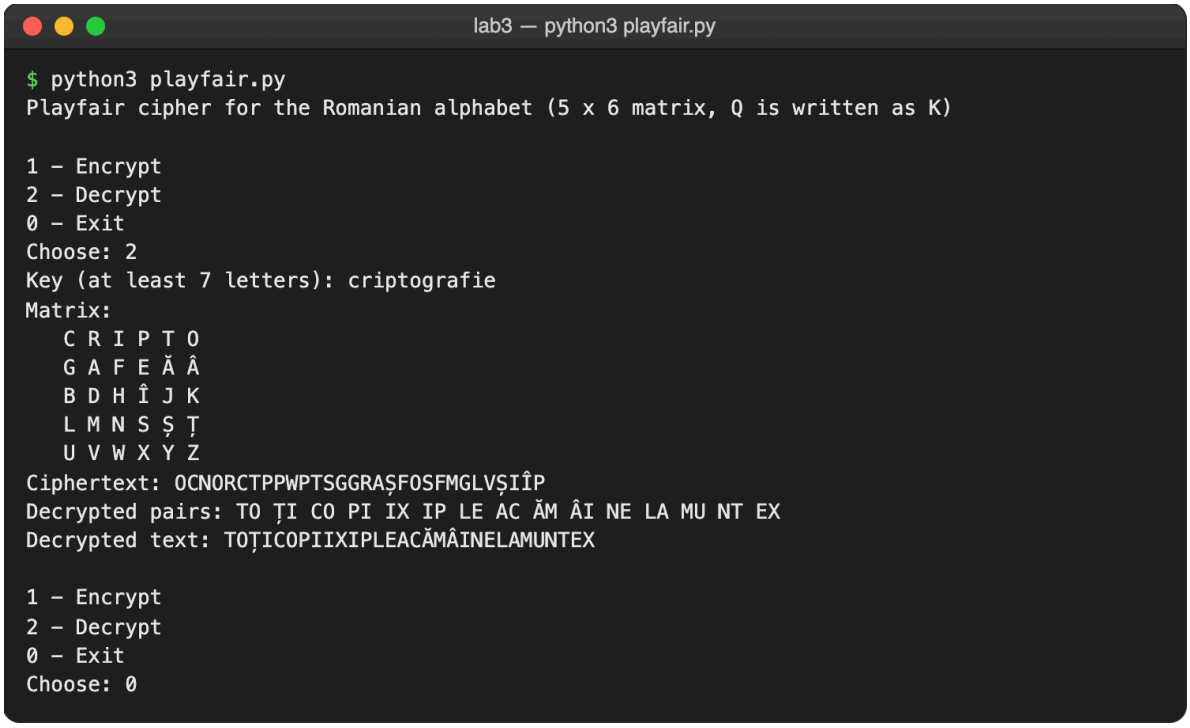


Figure 2. *Decryption of the ciphertext with the same key*

Decrypting `OCNORCTPPWPTSGGRAȘFOSFMGLVȘIÎP` with the same key gives back the pairs `TO ȚI CO PI IX IP LE AC ĂM ÂI NE LA MU NT EX`, so the program outputs

`TOȚICOPIIXIPLEACĂMÂINELAMUNTEX`

As the task says, the final step is done manually. Reading the text, I split it into words: `TOȚI COPIIXIP LEACĂ …` does not make sense, so the X between the I's is a filler and is removed (COPI-II), and the X at the end is the filler for the odd length and is removed too. The result is:

TOȚI COPIII PLEACĂ MÂINE LA MUNTE

which is the original message. This step cannot be fully automatic, because an X in the decrypted text can also be a real letter of the message (for example in "TEXT"), and only someone who understands the language can decide.

4.4. Invalid input

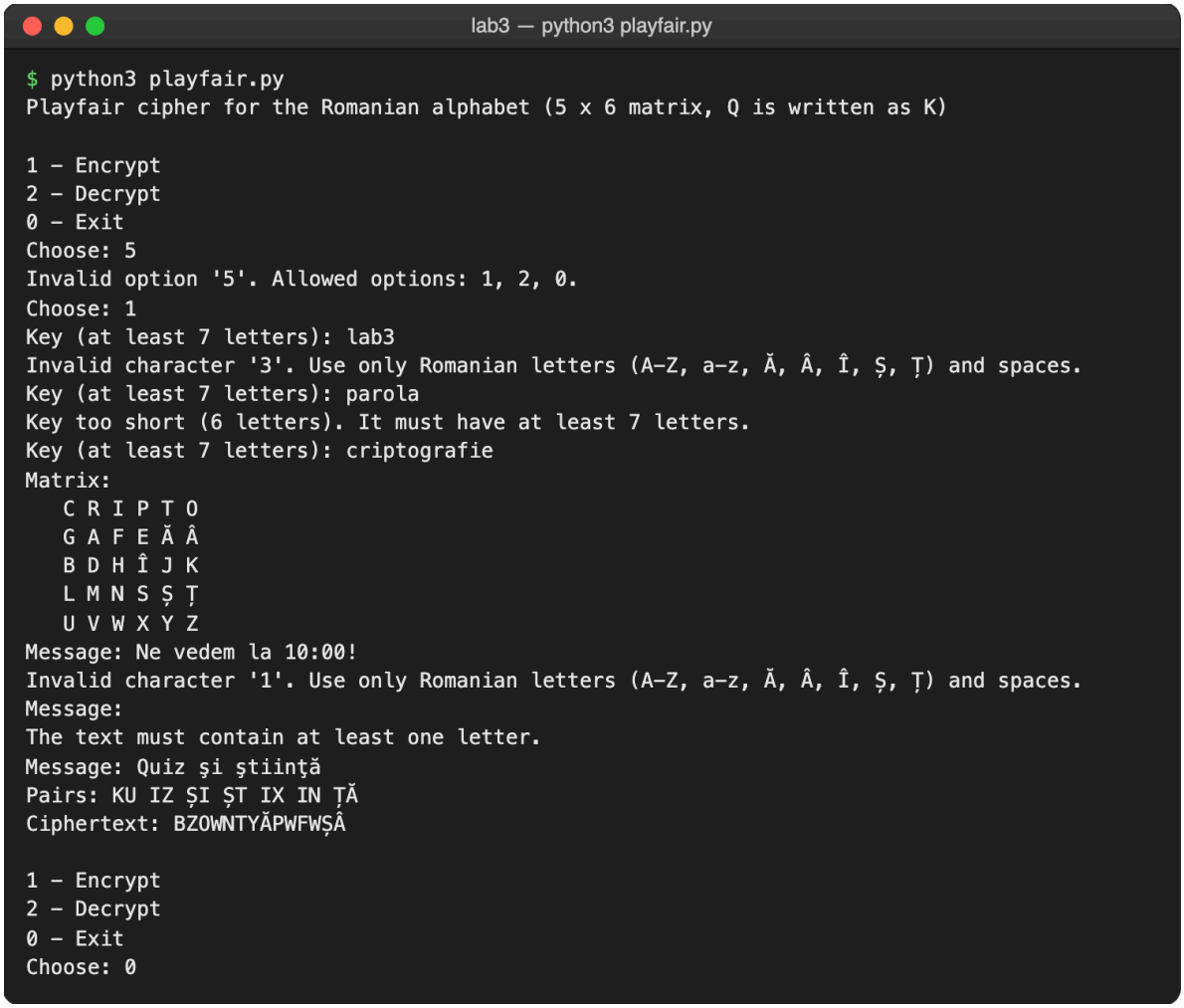


Figure 3. *Invalid menu option, invalid keys and invalid messages during encryption*

In Figure 3 the menu option 5 is rejected, the key “lab3” is rejected because of the digit 3 (the message shows the allowed range of characters), and “parola” is rejected because it has only 6 letters. The message “Ne vedem la 10:00!” is rejected because of the digit 1, and a message made only of spaces is rejected as empty. The last message, “Quiz şi ştiinţă”, is valid: it is typed with the cedilla letters ş and ţ and contains a Q. The program converts them to Ș, Ț and K, so the pairs are `KU IZ ȘI ȘT IX IN ȚĂ`.

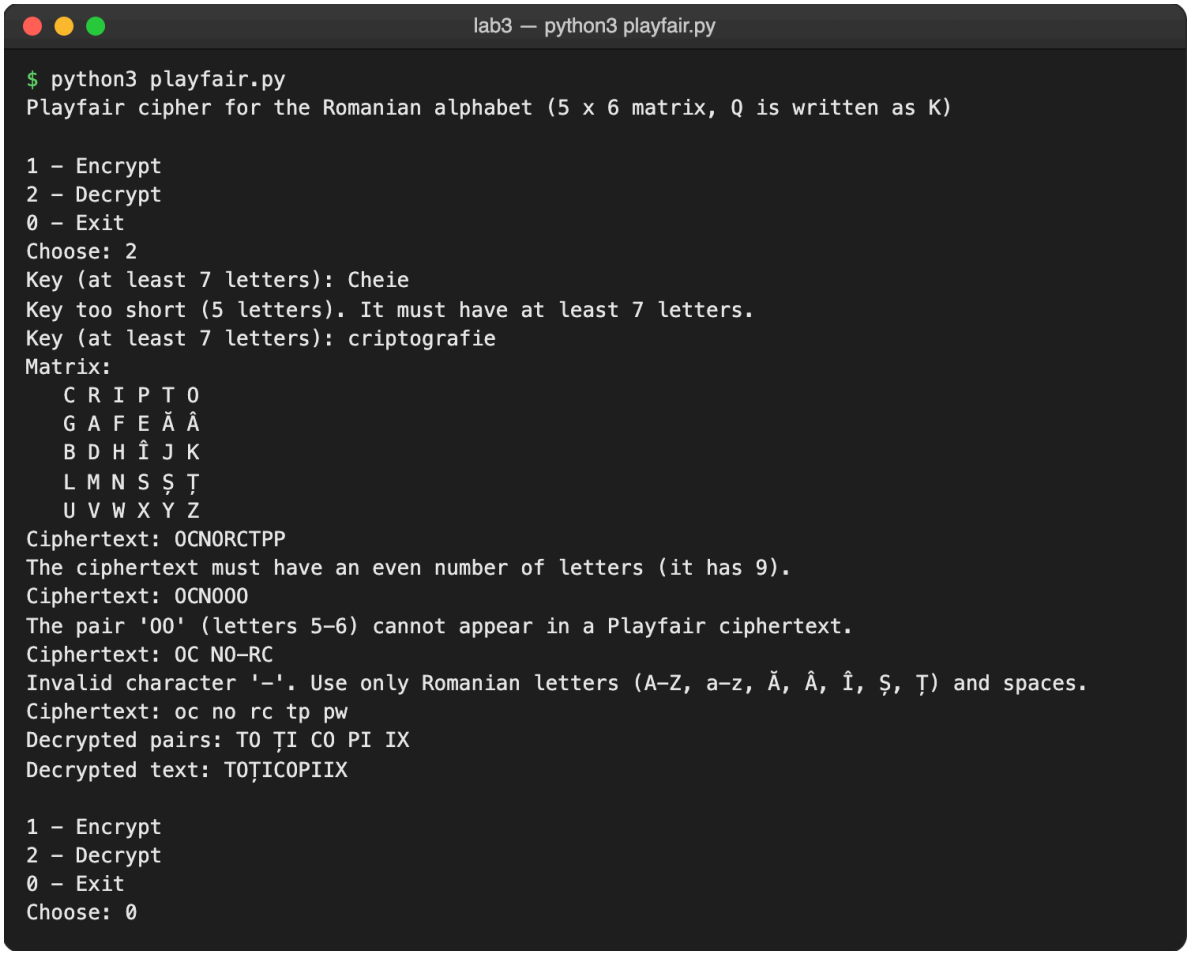


Figure 4. *Invalid key and invalid ciphertexts during decryption*

In Figure 4 the key “Cheie” is too short. Then the ciphertext `OCNORCTPP` is rejected because it has 9 letters, `OCNOOO` because the pair “OO” can never appear in a Playfair ciphertext, and `OC NO-RC` because of the character “-”. The last ciphertext is written in lowercase with spaces, which is accepted, and it decrypts to the first five pairs of my message, `TOȚICOPIIX`.

5. Conclusions

In this laboratory work I implemented the Playfair cipher for the Romanian alphabet. The main difficulty was that the Romanian alphabet has 31 letters, which is a prime number and does not fit into a rectangular matrix. I solved it with a 5×6 matrix without the letter Q, the rarest Romanian letter, which is written as K, in the same way as J and I are merged in the English Playfair. This keeps the ciphertext made only of letters, as the task requires. I checked my implementation with the example from the lab description, and it produced exactly the same 230-letter ciphertext.

Compared to the monoalphabetic cipher from Lab 2, Playfair is clearly stronger. In my example the letter E was encrypted as G, F and Î, T as O and I, and I as O and P, depending on the other letter of the pair, so counting single letters no longer reveals the key. The key space is also huge (about $29! \approx 8.8 \cdot 10^{30}$ different keys for the 5×6 matrix). However, Playfair is still a substitution, only of pairs of letters instead of single letters: the same pair is always encrypted the same way, the frequencies of pairs are preserved, reversed pairs give reversed results, and a letter is never encrypted into itself. With a long enough text and a computer it can be broken in seconds, so today it is only of historical and educational interest.

I also understood why decryption needs a human at the end: the filler letters and the missing spaces can only be corrected by someone who understands the message, because a filler X looks exactly like a real X. Doing this lab helped me see how the idea of encrypting blocks instead of single letters, which modern block ciphers also use, makes frequency analysis much harder.

6. Source code

The full source code (`playfair.py`) is available in the GitHub repository:

https://github.com/httpcatalin/cs-labs (folder `lab3`)